

SecureMedi Project Flow

High-Level Flow

1. Frontend (Next.js) calls backend APIs (FastAPI).
2. Backend handles authentication, admin/doctor/patient logic, wallet management, audit logs, and optional blockchain writes.
3. Supabase is used for doctors/patients records, while local JSON files support wallet and assignment state.

Startup Flow

1. Start backend from backend/app/main.py.
2. Middleware loads: authentication and audit.
3. Services initialize lazily (wallet service, blockchain service).
4. Start frontend from frontend/pages and connect to backend base URL from frontend/lib/api.ts.

Authentication Flow

1. User logs in from frontend/pages/login.tsx.
2. Backend verifies identity in backend/app/api/auth_routes.py and backend/app/services/auth_service.py.
3. JWT tokens are issued and validated by backend/app/services/jwt_service.py.
4. Protected routes are checked by backend/app/middleware/auth.py.

Admin Flow

1. Admin UI lives in frontend/pages/admin.tsx.
2. CRUD operations for doctors/patients use backend/app/api/admin_routes.py.
3. Doctor create path:
 - Auto-generate wallet if missing via backend/app/services/wallet_service.py.
 - Persist doctor in Supabase via backend/app/services/supabase_service.py.
 - Optionally register on blockchain via services/blockchain_service.py.
4. Wallet status uses GET /api/admin/wallets/available backed by wallets_assigned.json.

Doctor Flow

1. Doctor dashboard is frontend/pages/dashboard.tsx.
2. Assigned patients, vitals, and alerts are fetched via doctor and health routes.
3. Emergency access is managed by backend/app/services/emergency_service.py and related doctor routes.

Data Flow

1. Supabase stores doctors and patients (service layer: backend/app/services/supabase_service.py).

2. Local files track runtime assignment state:

- doctor_patient_assignments.json
- wallets_assigned.json

3. Optional logs are written under logs.

Blockchain Flow

1. Backend connects to GANACHE_URL from .env using services/blockchain_service.py.
2. Contract ABI is loaded from contracts/abi.json.
3. Registration and access-log operations flow through blockchain service wrappers.

Audit and Security Flow

1. Audit middleware captures protected endpoint activity.
2. Audit events are persisted by backend/app/services/audit_service.py.
3. Security hardening references:
 - documentation/SECURITY.md
 - documentation/INCIDENT_ROTATION_CHECKLIST.md

End-to-End Example (Admin Creates Doctor)

1. Admin submits doctor form in frontend/pages/admin.tsx.
2. Backend endpoint /api/admin/registry/doctors validates payload.
3. Wallet is assigned/generated (if needed).
4. Doctor record is inserted in Supabase.
5. Optional blockchain registration executes.
6. Audit event is recorded.
7. Frontend receives success + wallet details and refreshes wallet status.